Dynamos at ArGuard Shared Tasks: Multimodal Transformers and Selective Ensembling for Binary and Fine-Grained Arabic Hateful Meme and Harmful Prompt Detection

Sheikh Abdul Wahid
Universidade do Algarve
a91643@ualg.pt

Muhammad Yamin
University of Girona
u6110888@udg.edu

Ali Arshad
Skolkovo Institute of Science and Technology
Ali.Arshad@skoltech.ru

Abstract

For the ArGuard 2026 shared task on harmful content detection in Arabic memes (Task A) and LLM prompts (Task B), we fine-tune Arabic transformer encoders across all four subtasks. Memes additionally use a vision tower (ViT-B/16 for the binary subtask, CLIP ViT-L/14 for the fine-grained one) late-fused with MARBERTv2; ensembling choices and decision thresholds are tuned per subtask throughout. Two methodological choices account for much of our gains: selecting checkpoints by development macro-F1 rather than loss, and aligning the B2 training target with the evaluation distribution. Our largest single effect, however, was diagnostic. The B1 blind test set is approximately class-balanced while its training data is ∼80/20, so our dev-tuned threshold collapsed macro-F1 from 0.9557 on dev to 0.6345 on test. We recovered the test class balance algebraically from the aggregate metrics the platform reported to all participants; re-thresholding the *same* saved probabilities, with no retraining, restored 0.8628. Because this uses only aggregate leaderboard feedback (within the task rules), the B1 figure is not directly comparable to teams that did not use the channel and is not a generalisation estimate. This correction is also not general: on the weaker multi-label classifier (A2), stronger prior matching reduced performance in every setting we tested, so matching the test prior is not necessarily F1-optimal. We additionally report a catalogue of quantified negative results, including external-data augmentation with AraSafe. Final test macro-F1: A1 0.7869, A2 0.3463, B1 0.8628, B2 0.7622.

1 Introduction

The ArGuard 2026 shared task (Alam et al., 2026) covers two tracks: Task A, hateful-meme detection over the AHA-Memes benchmark (Kmainasi et al., 2026) with binary (A1) and fine-grained multi-label (A2) subtasks; and Task B, harmful-prompt detection with binary safe/unsafe (B1) and harm-domain (B2) subtasks.[1] All subtasks are scored by macro-F1.

Our guiding principle was *cheap evidence before expensive compute*: most design choices were A/B-tested on the development set, and negative results were retained and are reported here. Our contributions are: (1) a diagnostic showing that the aggregate metrics reported by the evaluation platform over-determine the binary confusion matrix, exposing the blind test set's class balance without access to any individual test label (Section 6, derivation in Appendix C); (2) evidence that prior correction rescues a strong classifier under label shift (+22.8 points on B1) yet damages a weak one (A2) in every setting we tested; and (3) a catalogue of quantified negative results (Section 8).

2 Related Work

Arabic harmful-content detection. Arabic offensive- and hate-speech detection has been driven by dedicated shared tasks and by dialect-aware pretrained encoders such as AraBERT (Antoun et al., 2020), MARBERT (Abdul-Mageed et al., 2021), and CAMeLBERT (Inoue et al., 2021), which we build on. ArGuard 2026 (Alam et al., 2026) extends this line to two harder settings: multimodal memes over the AHA-Memes benchmark (Kmainasi et al., 2026) and LLM harmful-prompt detection over dialectal Arabic (ArabicDialectSafety; Zaghouani et al., 2026).

Multimodal hateful memes. Meme classification couples OCR text with image content; the Hateful Memes Challenge (Kiela et al., 2020) popularised late- and early-fusion transformer baselines. We adopt late fusion of an Arabic text encoder with a vision tower, and find that the image encoder's pretraining alignment matters more than its ImageNet accuracy.

[1] Code, including the solver of Appendix C: https://github.com/alpha-31/arguard-2026

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 371–378
October 28-29, 2026 ©2026 Association for Computational Linguistics

Label shift and prior correction. Adjusting a fixed classifier to a new class prior is well established: prior-probability re-scaling (Saerens et al., 2002), the cost-sensitive/threshold view (Elkan, 2001), black-box label-shift estimation (Lipton et al., 2018; Garg et al., 2020), and the known form of the F1-optimal threshold (Lipton et al., 2014). Extracting information from aggregate leaderboard feedback is also studied: the Ladder analyses leaderboard overfitting (Blum and Hardt, 2015), and Whitehill (2016) recovers label information from a reported score. Our contribution is not the correction itself but the specific observation that, for a *binary* task, the accuracy and macro-precision a leaderboard reports *over-determine* the test confusion matrix, so the test prior – and the needed correction – can be recovered in closed form with no access to individual labels.

Gap. Most system-description papers emphasise architecture. We instead show that, across these four subtasks, aligning the system to the *evaluation target* – the training distribution, the checkpoint-selection metric, and the test prior – moved scores more than any model change, and we quantify where each technique, including several that failed, helped or hurt.

3 Data

Task A provides 3,500 training and 500 development memes (OCR text plus image); Task B provides 16,835 training and 2,405 development prompts across six dialect groups, ∼80/20 unsafe/safe, drawn from the ArabicDialectSafety benchmark (Zaghouani et al., 2026). Task A additionally includes a held-out 500-meme *dev_test* split, on which the development-phase leaderboard scores Task A submissions, and a 500-meme blind test set for the final evaluation phase; Task B's blind test set contains 7,809 prompts. In the official B2 data, *safe* prompts also carry topic-style granular labels and the leaderboard scores over all rows, so training on unsafe-only rows – the intuitive reading of the task – mistrains for the actual target; we therefore train on all rows (its effect is quantified in Section 5).

4 System

4.1 Task A: memes

We use late fusion: a MARBERTv2 text encoder (Abdul-Mageed et al., 2021) and a vision tower are each pooled, concatenated, and passed to a two-layer MLP head. We compare two vision towers – an ImageNet-pretrained ViT-B/16 (Dosovitskiy et al., 2021) and the image–text-aligned CLIP ViT-L/14 (Radford et al., 2021). A1 is a single fusion model; A2 is a soft-average ensemble of the CLIP fusion and three ViT seeds with per-class sigmoid thresholds tuned on dev. Checkpoints are selected by development macro-F1 rather than by loss. For the final evaluation phase we retrain the frozen recipes on train+dev with no further tuning. Because the dev-tuned CLIP-based A1 configurations did not transfer to the held-out dev_test split (Section 5), the final A1 recipe uses the simpler ViT-B/16 tower (Appendix A); A2, which did transfer, retains CLIP.

4.2 Task B: prompts

B1 and B2 fine-tune Arabic encoders (MARBERTv2, AraBERTv02 (Antoun et al., 2020), CAMeLBERT-mix (Inoue et al., 2021), and AraBERT-large) with inverse-frequency-weighted cross-entropy. B1 is a single MARBERTv2 with a tuned decision threshold; ensembling hurt B1 on both dev and the blind test (Section 5), so the final B1 system is a single model. B2 is an all-rows-trained (Section 3) four-model soft ensemble decoded by argmax.

5 Results

Setup. All four subtasks are scored by macro-F1. We tune on the development set, select checkpoints by development macro-F1, and report development, development-phase leaderboard, and blind-test scores (Table 1); full hyperparameters and hardware are in Appendix A.

Official runs. Our official (final) submission per subtask is the bold *Test* value in Table 1: A1, the single ViT-B/16 fusion retrained on train+dev (0.7869); A2, the CLIP + three-seed ensemble (0.3463); B1, the single MARBERTv2 with the prior-corrected threshold (0.8628, Section 6); B2, the all-rows four-model ensemble (0.7622). Numbers obtained during the evaluation phase are so marked; all others (e.g. the A2 prior-matching variants and per-class breakdowns) are post-hoc scoring against the gold labels the organisers released after the phase closed.

Design choices that moved A1/A2. On A1 dev, the CLIP ViT-L/14 tower beat ImageNet ViT-B/16

Subtask	Dev	Dev-phase LB	Test
A1	0.7795	0.7050	**0.7869**
A2	0.3845	0.3223	**0.3463**
B1	0.9557	0.96	**0.8628**
B2	0.8968	0.90	**0.7622**

Table 1: Macro-F1 across phases. B1's test score is after prior correction (0.6345 before; Section 6). Dev-phase LB values are shown as displayed by the leaderboard.

Threshold	Class	P	R	F1
0.10 (dev-tuned)	safe	0.964	0.359	0.523
	unsafe	0.600	0.986	0.746
0.9985 (prior)	safe	0.865	0.865	0.865
	unsafe	0.861	0.861	0.861

Table 2: B1 per-class scores against the released gold labels, before and after prior correction. Macro-F1 0.6345 $\rightarrow$ 0.8628.

by 3 points as a single model (0.7702 vs. 0.7393; single seed on 500 dev memes, so treat as indicative), and selecting checkpoints by macro-F1 rather than loss added 2.9. This dev advantage did not survive: CLIP-based A1 configurations failed to transfer to dev_test, which is why the final A1 recipe uses ViT-B/16. The dev-tuned "mega" ensemble that looked best on dev (0.7795), however, did not transfer to the held-out dev_test split (0.7050, the development-phase leaderboard score); the final single-model ViT-B/16 recipe, retrained on train+dev, scored 0.7869 on the blind test set. The same pattern held for A2 (blind test 0.3463 vs. 0.3223 for the dev-tuned ensemble).

Design choices that moved B1/B2. Aligning B2 training with the scored target – all rows rather than unsafe-only (Section 3) – helps: a controlled TF-IDF proxy that isolates this single change improves macro-F1 by $+7.4$ (0.780 $\rightarrow$ 0.854). The corresponding development-phase leaderboard submissions rose 0.82 $\rightarrow$ 0.90, but that comparison also reflects other differences between the two runs, so we rely on the proxy for the isolated effect. On B1, ensembling hurt on both dev and the blind test (0.8165 ensemble vs. 0.8628 single), so the final B1 system is a single MARBERTv2.

Rankings. On the official leaderboards, our submissions ranked 4th of 16 teams on A1, 5th of 11 on A2, 4th of 22 on B1, and 5th of 17 on B2, above the official baselines (0.678, 0.295, 0.592, and 0.687, respectively).

6 Analysis: Distribution Shift and Prior Correction

B1 fell from 0.9557 (dev) to 0.6345 (test), a 32-point collapse for which we found no implementation error. The evaluation platform reports accuracy, macro precision, macro recall, and macro-F1 for every submission, and the evaluation phase permitted repeated submissions (20 per day, 100 in total), so this aggregate feedback was available to all participants on equal terms. We used it only to infer the aggregate class prior; no individual test label was accessed. We made 8 evaluation-phase submissions on B1 in total, 3 of which used the corrected threshold. The procedure is documented in full here; see also the Ethical Considerations section. For a binary task, these aggregates combined with our own predicted counts *over-determine* the confusion matrix (Appendix C). Solving:

	predicted	actual
unsafe	6,335 (81%)	3,851 (49%)
safe	1,474 (19%)	3,958 (51%)

The recovered balance reproduced the reported macro recall to four decimal places. Re-solving from our corrected submission gives a result consistent with the same 3,851/3,958, but this is a weaker check than the first: the solve is well-conditioned only when the predicted positive count $\hat{P}$ is far from $N/2$, and the corrected submission sits near $\hat{P} \approx N/2$ where the system is ill-conditioned (the recovered TP has an uncertainty of roughly ± 28 rather than ± 1; Appendix C), so we treat it as corroborating rather than independently confirmatory. **The organizers released the gold test labels after the evaluation phase closed, and the true distribution is exactly 3,851 unsafe / 3,958 safe**, confirming the recovered values with no error. Train and dev are $\sim$80/20 unsafe; the blind test set was rebalanced.

Table 2 shows the mechanism. Our dev-tuned threshold (0.10 on *unsafe*) buys near-perfect unsafe recall at the cost of missing two-thirds of the safe class; because macro-F1 weights both classes equally, the collapse follows. Note that the *safe* class is not poorly modelled: re-thresholding the same probabilities lifts its F1 to 0.865 (Table 2). Under the dev-tuned threshold, the model's rare safe predictions are precise (0.964); the class is simply almost never predicted.

We refer to the fix as *prior-informed threshold adjustment* ("prior correction" for short): the clas-

Subtask	P	R	Gap	Indication
B1 before	0.782	0.673	0.109	prior mismatch?
B1 after	0.863	0.863	0.000	prior matched
B2	0.770	0.769	0.001	balanced
A1	0.781	0.794	0.013	near-balanced

Table 3: Macro precision/recall gap as a cheap *flag* for threshold imbalance (it suggests, and does not by itself establish, a prior mismatch; the solve of Section 6 is what confirms it). P/R are CodaBench-reported values; the B1 and A1 test-set balances subsequently solved from them are confirmed exactly against released gold labels.

sifier is unchanged, and only the decision threshold is set so that the predicted class prevalence matches the inferred test prevalence. Re-thresholding the same saved probabilities so that 49.3% of rows are predicted unsafe, with no retraining and no new data, moved B1 from 0.6345 to 0.8628 (Figure 1), submitted within the same evaluation phase under the same submission limits as every other run. Afterwards, the platform-reported accuracy, precision, recall, and macro-F1 all agreed to four decimals with the values implied by the reconstructed confusion matrix, an independent consistency check on the correction.

The organizers subsequently released gold labels for all four subtasks. Each figure in this section that was originally an inference has since been checked against them: the released B1 test balance is exactly 3,851/3,958, and the A1 class balance recovered algebraically from the same binary confusion-matrix solve (148 Hateful / 352 Not Hateful, discussed below) is likewise exact against the released A1 gold. No correction to any reported number was required.

A cheap first check. The macro precision–recall gap flags the failure without solving the matrix (Table 3). The gap indicates a threshold imbalance, which can arise for several reasons; a large gap *suggests* prior mismatch rather than proving it, and the solve above is what confirms it.

A1's small 0.013 gap suggested that no correction was needed: solving its confusion matrix from CodaBench's four reported aggregates gives 148 true Hateful memes against 161 predicted (29.6% vs. 32.2%), too close to be worth adjusting. The released A1 gold confirms 148 exactly.

Prior correction is not a general remedy. Applying the same correction to A2 *hurt*, and in each of the three settings we tested, stronger matching reduced performance: 0.3463 (original, 2.41 predicted labels/meme) $\rightarrow$ 0.3335 (per-class rates approximately matched, 1.63) $\rightarrow$ 0.3097 (per-class rates exactly matched, 1.39 labels/meme, the true overall rate). All three variants were scored by the official platform during the evaluation phase. This is consistent with the known geometry of F1: for calibrated probabilities, the F1-optimal decision threshold equals half the maximum achievable F1 (Lipton et al., 2014). Matching the predicted positive rate to the test prior is therefore not guaranteed to maximize F1. More precisely, the operative quantity is the *achievable per-class* F1 (Lipton et al., 2014), not model size or overall accuracy: where a class's attainable F1 is high (B1, per-class F1 ≈ 0.86) matching the prior helps, whereas for low-attainable-F1 classes (A2 spans 0.06–0.66) over-prediction buys far more recall than it costs precision, so prior matching hurts (the *Exclusion* class is a stark example; Section 7). We use "strong"/"weak" loosely for this; a sharper test would compute each class's F1-optimal threshold directly, which we leave to future work.

Saturation. Fine-tuned classifiers push nearly all probability mass to 0 or 1: on B1, thresholds 0.1 and 0.9 differ in only 353 of 7,809 predictions (4.5%), so a conventional 0.05–0.95 threshold grid explores a sliver of the operating curve. Threshold search should be defined over probability *quantiles*; model averaging also de-saturates the distribution (55.2% of a 4-model ensemble's rows exceed 0.99 vs. 76.6% for a single model).

7 Error Analysis

B2 (harm domains). The weakest B2 class is *Bully* (test-set F1 0.529, recall 0.416 against the released gold). The model is precise when it commits (precision 0.724) but confuses most instances with the neighbouring *harassment* and *harm-to-others* categories, which together make up 31% of the B2 test set. This is a semantic-overlap failure between adjacent harm types, not a calibration one, and is the main ceiling on B2 macro-F1.

A2 (fine-grained memes). A2 errors are dominated by data sparsity rather than thresholding. *Exclusion* has 10 training and 3 test instances; our system predicted it 94 times for an F1 of 0.062, and no decision rule recovers an essentially unobserved class. Per-class F1 spans 0.06–0.66, so the macro average is set by these rare classes, which is why

Attempt	Effect
AraSafe augm., full 24k (B1)	$0.926 \rightarrow 0.888$
AraSafe, unsafe-only 13.5k (B1)	$0.926 \rightarrow 0.917$
AraSafe, mapped classes (B2)	$0.854 \rightarrow 0.850$
Dialect token feature	B1 -0.4, B2 -0.1
Arabic normalization	A1 -1.3, B1 -0.3
Ensembling B1 (test)	$0.863 \rightarrow 0.817$
Dropping the collapsed model from the B2 ensemble (test)	$0.7622 \rightarrow 0.7624$
Prior correction on A2 (test)	$0.346 \rightarrow 0.310$
Image augmentation (A1)	$+0.0006$ (noise)
5-tower vision ensemble (A1, test)	$0.787 \rightarrow 0.757$

Table 4: Quantified ablations and negative results (macro-F1). Rows with arrows read baseline $\rightarrow$ variant; the remaining rows report the change alone. Effects are on dev unless marked (test). Among the (test) rows, the B1-ensembling and A2 prior-correction results are *evaluation-phase* submissions; the 5-tower and drop-collapsed-model results are *post-hoc* scoring against the gold labels released after the phase closed. The 5-tower ensemble soft-averages SigLIP-so400m-384, CLIP ViT-L/14-336, CLIP ViT-L/14, DINOv2-large, and CLIP+AraBERT fusions, compared against the single frozen A1 recipe on the blind test set.

prior matching (Section 6) hurts rather than helps here.

B1 (safe/unsafe). Under the dev-tuned threshold the model's *safe* predictions are highly precise (0.964) but cover only a third of the class (Table 2); the *safe* class is well modelled but almost never predicted. This is precisely the label-shift failure diagnosed and corrected in Section 6, and it is invisible in the headline macro-F1 until the per-class scores are inspected.

8 Ablation and Negative Results

Table 4 summarizes the main techniques we evaluated, each with its measured effect. We highlight three. **External data:** AraSafe (Mubarak et al., 2025) augmentation hurt in every variant we tried despite zero prompt overlap with ArGuard data; ArGuard's safe prompts are deliberately borderline, while AraSafe's are plainly benign, so augmentation appears to teach the wrong boundary. In our experiments, matching the training distribution to the evaluation target mattered more than data volume. **Training instability:** AraBERT-large, the weakest encoder on every subtask where we evaluated it, collapsed outright on B2 under our recipe (lr 1e-5, batch 16): training loss sat flat at $\approx 1.97 \approx \ln 7$, i.e. uniform guessing over seven classes, with 100% of predictions in one class. The collapse was invisible in the aggregate ensemble output and caught only by auditing per-model predicted distributions; we do not claim model size alone caused it. **Surface normalization** reduced performance, suggesting that orthographic cues such as character elongation carry useful signal for dialect-pretrained encoders. A Qwen2.5-VL-7B LoRA fine-tune (Bai et al., 2025; Hu et al., 2022) reached A1 0.7232 / A2 0.2535, below the tuned discriminative ensembles at this data scale.

9 Conclusion

Across four subtasks, the decisions that moved scores most were about the *evaluation target*, not the architecture: aligning B2's training distribution with what the leaderboard scores (+8), selecting checkpoints by macro-F1 (+2.9), and detecting that B1's blind test set was approximately class-balanced while its training data was not (+22.8, from re-thresholding alone). Aggregate leaderboard metrics can carry information about the test distribution beyond the final score: for a binary task they expose the test class balance without access to any individual label. We recommend the macro precision–recall gap as a cheap first check for threshold imbalance, and we caution against reflexive prior matching: the same correction that rescued a strong classifier damaged a weak one in every setting we tested.

Limitations

Our headline B1 result (0.8628) is obtained by re-thresholding to a test prior recovered from aggregate leaderboard feedback. Although this is disclosed and within the task rules, it means the B1 figure is not directly comparable to teams that did not use the channel, and – because the threshold is fitted to the test prior rather than estimated independently – it is not a generalisation estimate. Our distribution-shift solver also applies exactly only to binary subtasks; for multi-label tasks we could only match observed per-class rates, which we showed is actively harmful for weak classifiers. The main limitation for A2 is data sparsity rather than threshold calibration: *Exclusion* has 10 training and 3 test instances, and no decision rule recovers an essentially unobserved class. Class-conditional synthetic augmentation is a natural next experiment; we did not have time to evaluate it rigorously. We do not report confidence intervals for the ablations, as the final evaluation is a single shared-task test set and

most dev-phase comparisons rest on single runs. All findings are from a single shared task in Arabic; the F1 threshold argument is language-independent, but the magnitude of the effects may not transfer.

Ethical Considerations

This work targets *detection* of harmful content to support moderation of Arabic online spaces, a comparatively under-resourced setting. We used only the official ArGuard data and the publicly released AraSafe corpus (Mubarak et al., 2025) (disclosed per task rules; it did not enter any final system). Harmful-content classifiers can encode dialect- or community-correlated biases; our per-class error analysis (weak minority classes) suggests deployment should keep a human in the loop, particularly for rare categories.

Our diagnostic in Section 6 also constitutes a disclosure: reporting multiple aggregate metrics during a blind evaluation phase reveals the test class balance of a binary task to any participant. We recommend that future evaluations report only the ranking metric during the blind phase, or round auxiliary aggregates coarsely enough that the confusion-matrix solve of Appendix C becomes ambiguous.

References

Muhammad Abdul-Mageed, AbdelRahim Elmadany, and El Moatez Billah Nagoudi. 2021. ARBERT & MARBERT: Deep bidirectional transformers for Arabic. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 7088–7105, Online. Association for Computational Linguistics.

Firoj Alam, Md. Rafiul Biswas, Mohamed Bayan Kmainasi, Ali Ezzat Shahroor, Hamdy Mubarak, Georgios Mikros, Abul Hasnat, and Wajdi Zaghouani. 2026. ArGuard Shared Task: Harmful content detection in Arabic Memes and LLM Prompts. In *Proceedings of the Fourth Arabic Natural Language Processing Conference: Shared Tasks*, Budapest, Hungary. Association for Computational Linguistics.

Wissam Antoun, Fady Baly, and Hazem Hajj. 2020. AraBERT: Transformer-based model for Arabic language understanding. In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools*, pages 9–15, Marseille, France. European Language Resources Association.

Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, and 8 others. 2025. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*.

Avrim Blum and Moritz Hardt. 2015. The ladder: A reliable leaderboard for machine learning competitions. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 1006–1014.

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*.

Charles Elkan. 2001. The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 973–978.

Saurabh Garg, Yifan Wu, Sivaraman Balakrishnan, and Zachary C. Lipton. 2020. A unified view of label shift estimation. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Go Inoue, Bashar Alhafni, Nurpeiis Baimukan, Houda Bouamor, and Nizar Habash. 2021. The interplay of variant, size, and task type in Arabic pre-trained language models. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, pages 92–104, Kyiv, Ukraine (Virtual). Association for Computational Linguistics.

Douwe Kiela, Hamed Firooz, Aravind Mohan, Vedanuj Goswami, Amanpreet Singh, Pratik Ringshia, and Davide Testuggine. 2020. The hateful memes challenge: Detecting hate speech in multimodal memes. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Mohamed Bayan Kmainasi, Ali Ezzat Shahroor, Abul Hasnat, Md. Rafiul Biswas, Wajdi Zaghouani, and Firoj Alam. 2026. AHA-Memes: A fine-grained multimodal benchmark for understanding hate in arabic memes. *arXiv preprint arXiv:2607.27393*.

Zachary C. Lipton, Charles Elkan, and Balakrishnan Naryanaswamy. 2014. Optimal thresholding of classifiers to maximize F1 measure. In *Machine Learning and Knowledge Discovery in Databases (ECML PKDD 2014)*, pages 225–239. Springer.

Zachary C. Lipton, Yu-Xiang Wang, and Alexander Smola. 2018. Detecting and correcting for label shift with black box predictors. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 3122–3130.

Hamdy Mubarak, Abubakr Mohamed, and Majd Hawasly. 2025. AraSafe: Benchmarking safety in Arabic LLMs. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 9976–9992, Suzhou, China. Association for Computational Linguistics.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning*, pages 8748–8763.

Marco Saerens, Patrice Latinne, and Christine Decaestecker. 2002. Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure. *Neural Computation*, 14(1):21–41.

Jacob Whitehill. 2016. Exploiting an oracle that reports AUC scores in machine learning contests. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence (AAAI)*, pages 1345–1351.

Wajdi Zaghouani, Md. Rafiul Biswas, Kholoud Khalil Aldous, and Mabrouka Bessghaier. 2026. Arabic-DialectSafety: A dialect-aware benchmark for Arabic content safety classification. *arXiv preprint arXiv:2608.01291*.

A Hyperparameters

Shared: AdamW, weight decay 0.01, warmup ratio 0.06, linear decay, max sequence length 128, seed 42 unless stated, bf16. Dev-phase checkpoint selection by dev macro-F1; final-phase runs fold dev into training with a fixed epoch budget and no selection. All runs: one A100 80GB; base encoders train in 5–10 minutes.

Qwen2.5-VL-7B LoRA (the generative comparison of Section 8). 4-bit NF4 QLoRA, rank 16, $\alpha = 32$, dropout 0.05, on the attention (q, k, v, o) and MLP $(\text{gate}, \text{up}, \text{down})$ projections; learning rate 1×10^{-4}, 3 epochs, batch size 1 with gradient accumulation 8, bf16, image side capped at 512 visual tokens. The image, OCR text, and a fixed instruction are given to the model, which generates the label; this reached A1 0.7232 / A2 0.2535, below the tuned discriminative ensembles at this data scale.

The B2 ensemble averages softmax probabilities (not logits) across the four models, aligned by example id and by class name; all models share the same seven-class label set and the same class-weighted loss, and the averaged distribution is decoded by argmax with no further threshold tuning.

	A1	A2
Text encoder	MARBERTv2	MARBERTv2
Vision encoder	ViT-B/16	CLIP ViT-L/14
Epochs	2	5
Batch size	16	16
Encoder / head LR	2e-5 / 1e-4	2e-5 / 1e-4
Head dim / dropout	512 / 0.1	512 / 0.1
Decision	argmax	per-class thr.
Ensemble	none	CLIP + 3 seeds

Table 5: Task A (late fusion).

	B1	B2
Encoders	MARBERTv2	4 models
Epochs	3	3
Batch size	32	32 (16 large)
LR	2e-5	2e-5 (1e-5 large)
Loss	weighted CE	weighted CE
Rows	all	all (Section 3)
Decision	prior-corr. thr.	argmax of avg.

Table 6: Task B. "4 models": MARBERTv2, AraBERTv02, CAMeLBERT-mix, AraBERT-large.

B Decision Thresholds

A2 per-class sigmoid thresholds (dev-tuned): Contempt 0.70, Dehumanization 0.50, Exclusion 0.05, Humor 0.50, Incitement 0.50, Inferiority 0.55, Mocking 0.45, Other 0.50, Sarcasm 0.35, Slurs 0.45.

B1: dev tuning selected 0.10 on *unsafe* (test macro-F1 0.6345). During the evaluation phase the submitted system was instead re-thresholded by quantile so that 49.3% of test rows were predicted unsafe (the balance recovered in Section 6), giving an operating threshold of 0.9985 on this model's saturated output distribution and test macro-F1 0.8628.

C Recovering the Binary Test Class Balance

Let N be the number of scored rows and $\hat{P}$ the number of rows our submission predicted positive ($\hat{N} = N - \hat{P}$ predicted negative); both are known to the submitter. The platform reports accuracy a and macro precision p. With TP and TN the unknown counts of correct positive and negative predictions:

$$TP + TN = aN, \tag{1}$$

$$\tfrac{1}{2}\left(\frac{TP}{\hat{P}} + \frac{TN}{\hat{N}}\right) = p. \tag{2}$$

Two linear equations in two unknowns yield TP and TN, and the true number of positives follows

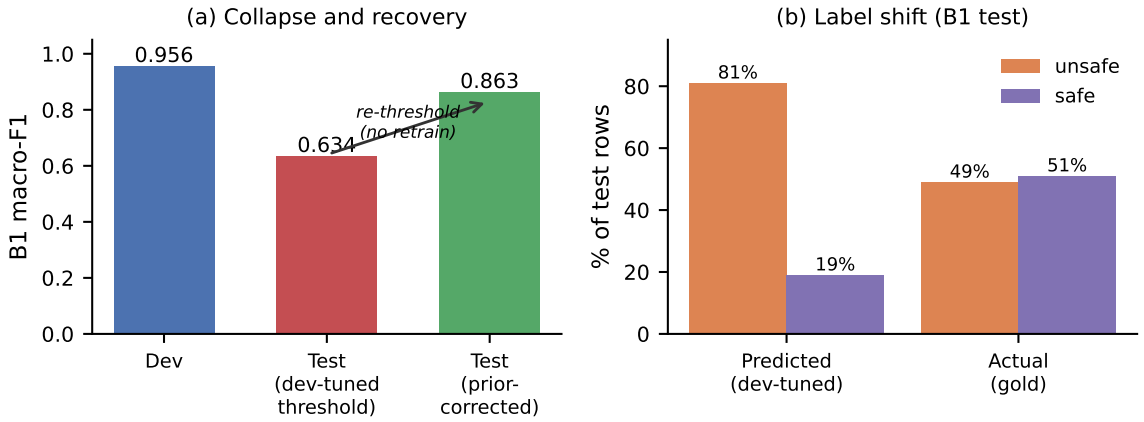


Figure 1: The B1 label-shift diagnostic (Section 6). **(a)** A dev-tuned threshold collapses blind-test macro-F1 from 0.956 to 0.634; re-thresholding the *same* saved probabilities to the recovered test prior restores 0.863, with no retraining. **(b)** The cause: the dev-tuned model predicts 81% unsafe, mirroring the ∼80/20 training prior, but the blind test set is approximately balanced (49/51). The test balance was recovered algebraically from the platform's aggregate metrics (this appendix) and later confirmed exactly against the released gold labels.

as $U = TP + (\hat{N} - TN)$. The reported macro recall is not needed for the solve and therefore serves as an independent over-determination check:

$$r = \tfrac{1}{2}\left(\frac{TP}{U} + \frac{TN}{N-U}\right). \tag{3}$$

The platform reports a, p, and r rounded to four decimal places, so the solved TP and TN need not be exact integers; we round them to the nearest integers.

Conditioning. The solve inverts a 2×2 system whose sensitivity scales with $\frac{1}{\hat{P}} - \frac{1}{\hat{N}}$; this vanishes as $\hat{P} \to N/2$, so the recovered counts are sharp when the submission's predicted positive rate is far from balanced and increasingly uncertain near it. For our first B1 submission ($\hat{P} = 6{,}335$ of $N = 7{,}809$) the rounding-level uncertainty in TP is $\approx \pm 1$; for a submission near $\hat{P} \approx N/2$ it is $\approx \pm 28$. We therefore rely on the first, well-conditioned submission for recovery: there, Eq. (3) reproduces the reported macro recall to four decimals, and the subsequently released gold labels confirm $U = 3{,}851$ exactly. Solves from near-balanced submissions are consistent with this but corroborative rather than independent confirmations.